

Traffic Signal Control Using Reinforcement Learning

Final Project Report

Introduction to Reinforcement Learning

Dr. Teddy Lazebnik

Students' Names and IDs:

Silin Michael - ID 327946109

Anne Claude Abinader - ID 331181909

Abstract

City traffic jams waste many hours and a huge amount of fuel every year.

The main issue is that most traffic lights run on simple fixed timers. So they can't adapt when traffic gets heavy or uneven in different directions.

In this project, we apply three model-free reinforcement learning (RL) algorithms to create a smart traffic light controller for a simulated four-way intersection. We set the problem up as a Markov Decision Process (MDP). For the state space, we tracked the lengths of the car queues and the current color of the traffic light. Using this setup, we trained:

1. Q-Learning
2. SARSA
3. Deep Q-Network (DQN)

agents, running them over multiple independent seeds to ensure our results were solid. All three of our RL agents clearly beat the traditional baseline methods.

We compared them against the best standard rule which is the Longest Queue First (LQF) controller. And against it our trained DQN agent improved the average reward by 17% (-896 vs. -1074) and dropped the average number of waiting cars per step from 3.62 to 3.02.

When compared to basic fixed-timer lights, our improvement reached an impressive 74%.

We also tested four different reward functions across five runs to see what helped the AI learn best. We found that giving rewards based on traffic throughput did the best job of keeping the lines short. Finally, we searched for the best hyperparameters for the learning rate α and discount factor γ , confirming that moderate values ($\alpha = 0.1$, $\gamma = 0.95$) gave the best results.

Overall, this project shows that RL agents can learn highly effective, adaptable traffic control strategies that consistently beat traditional rule-based controllers.

Contents

1	Introduction and Project Overview	3
1.1	Motivation	3
1.2	Objective	3
1.3	Scope	4
2	Problem Formulation	4
2.1	State Space (S)	4
2.2	Action Space (A)	5
2.3	Reward Function (R)	5
2.4	Environment Dynamics	5
3	Methodology	6
3.1	Reinforcement Learning Algorithms	6
3.1.1	Q-Learning (Off-Policy TD Control)	6
3.1.2	SARSA (On-Policy TD Control)	6
3.1.3	Deep Q-Network (DQN)	7
3.2	Baselines	7
3.3	Hyperparameter Tuning	8
3.4	Training Protocol	8
4	Results and Analysis	9
4.1	Training Convergence	9
4.2	Policy Evaluation	10
4.3	Hyperparameter Sensitivity	13
4.4	Reward Function Ablation Study	14
4.5	Value Function Analysis	16
5	Discussion and Limitations	17
5.1	Summary of Findings	17
5.2	Limitations	18
6	Conclusion	18

1 Introduction and Project Overview

1.1 Motivation

Traffic is a big problem in a lot of cities today, and people waste a lot of time sitting in traffic jams. A big reason for this is that most traffic lights use fixed timers. They just switch between green and red on a set schedule no matter how many cars are actually there.

This causes a lot of unnecessary delays. For example, a light might stay green for an empty street while a huge line of cars is stuck waiting at a red light on the other side. Reinforcement Learning can fix this problem. An RL agent looks at what is happening at the intersection right now, like how many cars are waiting in line and what color the light is. So by getting feedback from the environment, it learns the best times to switch the lights to keep the total waiting time as low as possible. Instead of just following hard-coded rules, the RL agent can learn on its own and adapt to traffic changes as they happen.

1.2 Objective

The objective of this project is to train and evaluate model-free RL agents to control a traffic signal at a single four-way intersection. with the goal of minimising total vehicle waiting time.

We implement and compare three algorithms spanning the tabular-to-deep spectrum:

1. Q-Learning (off-policy TD control)
2. SARSA (on-policy TD control)
3. Deep Q-Networks (DQN, neural function approximation)

We compared each of our AI models against four standard, hard-coded traffic rules, ranging from very simple timers to more advanced logic.

1.3 Scope

This project focuses on a single four-way intersection with North/South and East/West traffic. To make it realistic, the amount of traffic changes over time (like during rush hour) and is usually heavier in one direction than the other.

We trained all of our AI models completely from scratch. We didn't give them any built-in traffic rules or hints about how to solve the problem.

After training, we compared the models based on how well they performed, how fast they learned, and how tweaking their settings or rewards changed their behavior. This gives us a clear comparison of how both basic and advanced Reinforcement Learning algorithms handle the task.

2 Problem Formulation

We model the intersection control problem as a finite Markov Decision Process (S, A, R, P) .

2.1 State Space (S)

The state is a three element tuple:

$$s = (q_{NS}, q_{EW}, \phi)$$

- $q_{NS} \in \{0, 1, \dots, q_{\max}\}$: number of cars queued in the North/South direction (capped at $q_{\max} = 15$ in full training).
- $q_{EW} \in \{0, 1, \dots, q_{\max}\}$: number of cars queued in the East/West direction.
- $\phi \in \{0, 1, 2, 3\}$: the current signal phase. Phase 0 = NS green; Phase 1 = EW green; Phase 2 = NS yellow (transitioning to EW green); Phase 3 = EW yellow (transitioning to NS green).

The full-training state space size is $(q_{\max} + 1)^2 \times 4 = 16 \times 16 \times 4 = 1,024$ discrete states.

2.2 Action Space (A)

The action space is binary: $a \in \{0, 1\}$.

- $a = 0$ (Keep): maintain the current green signal.
- $a = 1$ (Switch): initiate a transition to the opposite green phase via a yellow intermediate step.

A *minimum green-time constraint* prevents the agent from switching more frequently than every 3 steps, avoiding destructive rapid toggling.

2.3 Reward Function (R)

The default reward at time step t is:

$$R_t = -(q_{NS} + q_{EW})$$

This penalises the total number of waiting vehicles and encourages the agent to keep queues as short as possible. In (Section 4.4), we also evaluate three alternative formulations:

$$R_t^{\text{max-queue}} = -\max(q_{NS}, q_{EW}) \quad (1)$$

$$R_t^{\text{quadratic}} = -(q_{NS}^2 + q_{EW}^2) \quad (2)$$

$$R_t^{\text{throughput}} = (d_{NS} + d_{EW}) - 0.1 (q_{NS} + q_{EW}) \quad (3)$$

where d_{NS}, d_{EW} are the number of departures (cars that pass through) in the current step.

2.4 Environment Dynamics

Arrivals. The environment simulates three time-varying traffic phases within each episode (length $T_{\max} = 300$ steps):

- Steps 1–30: *Morning Rush* - NS arrival probability $p_{NS} = 0.8$, EW $p_{EW} = 0.1$.
- Steps 31–70: *Mid-day* - $p_{NS} = 0.3$, $p_{EW} = 0.3$ (balanced).

- Steps 71–300: *Evening Rush* - $p_{NS} = 0.1$, $p_{EW} = 0.8$.

Departures. When a direction has the active green signal, 1 or 2 vehicles depart per step (chosen uniformly at random), provided the queue is non-empty.

No departures occur during yellow phases.

Termination. An episode ends after $T_{\max} = 300$ steps.

3 Methodology

3.1 Reinforcement Learning Algorithms

3.1.1 Q-Learning (Off-Policy TD Control)

Q-Learning estimates the optimal action-value function $Q^*(s, a)$ independently of the behaviour policy. After each transition (s, a, r, s') , the Q-table is updated as:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Because Q-Learning bootstraps from the *maximum* next-state value, it directly learns the greedy optimal policy even while exploring with ε -greedy behaviour. This off-policy character can cause slight overestimation in stochastic environments but generally leads to aggressive, efficient convergence.

3.1.2 SARSA (On-Policy TD Control)

SARSA (State-Action-Reward-State-Action) is the on-policy counterpart: it updates using the *actually chosen* next action a' :

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

Because the update target reflects the exploratory policy (including ε -random actions), SARSA tends to learn more conservative policies that are safer during exploration. In our environment, both algorithms converge to similar final performance, but SARSA’s on-policy nature leads to slightly more cautious early-training behaviour.

3.1.3 Deep Q-Network (DQN)

For problems with large or continuous state spaces, tabular Q-tables become impractical. DQN replaces the Q-table with a neural network $Q(s, a; \theta)$ parameterised by weights θ :

$$L(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

Two key stabilisation techniques are used:

- **Experience Replay:** transitions are stored in a replay buffer (capacity 30,000) and sampled in random mini-batches (size 128) to break temporal correlations.
- **Target Network:** a periodically-frozen copy θ^- of the main network stabilises the training targets, preventing harmful feedback loops.

Our network architecture is a fully-connected MLP: $6 \rightarrow 64 \rightarrow 64 \rightarrow 2$, with ReLU activations. The 6-dimensional input encodes $(q_{NS}/q_{max}, q_{EW}/q_{max}, \text{one-hot}(\phi))$, normalising queue lengths to $[0, 1]$ regardless of the maximum queue size. The target network is synchronised every 10 episodes.

3.2 Baselines

Four non-learning baselines are used for comparison:

- **Fixed-Time (5-step):** switches the signal every 5 steps, regardless of queue state simulating the simplest possible pre-timed controller.
- **Fixed-Time (10-step):** as above, but with a 10-step cycle.
- **Longest Queue First (LQF):** a greedy heuristic that switches the green phase to whichever direction has the longer queue, subject to the minimum green-time constraint. This represents a strong adaptive benchmark that has no learning overhead.

- **Random Switch:** selects action 0 or 1 uniformly at random at each step.

3.3 Hyperparameter Tuning

Before full-scale training, we conducted a grid search over the two most critical tabular hyperparameters using Q-Learning (500 training episodes, 50 evaluation episodes, $q_{\max} = 5$):

- Learning rate $\alpha \in \{0.01, 0.05, 0.1, 0.2\}$
- Discount factor $\gamma \in \{0.80, 0.90, 0.99\}$

The results, visualised as a heatmap in Figure 4, show that moderate values ($\alpha = 0.1$, $\gamma = 0.9\text{--}0.95$) yield the best performance. Very low α (slow learning) combined with high γ (strong future discounting) produced the worst results, as the agent could not accumulate sufficient updates within 500 episodes. Based on this analysis, the full training used $\alpha = 0.1$, $\gamma = 0.95$.

3.4 Training Protocol

Full-scale training was conducted with the following configuration:

Parameter	Q-Learning / SARSA	DQN
Episodes	20,000	2,500
Seeds	3 (42, 123, 777)	3 (42, 123, 777)
$q_{\max}$	15	15
$T_{\max}$	300 steps	300 steps
ϵ schedule	1.0 $\rightarrow$ 0.01 (decay 0.9997)	1.0 $\rightarrow$ 0.01 (decay 0.9982)
α	0.1	-
γ	0.95	0.95
DQN learning rate	-	5×10^{-4}
Replay buffer	-	30,000
Batch size	-	128
Eval episodes	200	200

4 Results and Analysis

4.1 Training Convergence

Figure 1 shows the mean $\pm$ standard deviation learning curves across 3 seeds for each algorithm, smoothed with a 500-episode moving average (50-episode for DQN).

All three algorithms exhibit clear, sustained improvement from initial random behaviour to a stable converged policy. Table 1 summarises the quantitative convergence statistics.

Table 1: Mean episode reward at early training (first 10%) and late training (last 10%) across 3 seeds. Standard deviation across seeds in parentheses.

Algorithm	Episodes	Early Reward	Late Reward
Q-Learning	20,000	$-3,104 (\pm 7)$	$-969 (\pm 3)$
SARSA	20,000	$-3,104 (\pm 9)$	$-974 (\pm 4)$
DQN	2,500	$-3,086 (\pm 63)$	$-947 (\pm 8)$

All algorithms improve by approximately **70%** from start to finish. The inter-seed standard deviation is very low (3–8 reward units), confirming strong reproducibility. Notably, DQN achieves a comparable late-training reward to the tabular methods in only 2,500 episodes - roughly $12.5\times$ fewer episodes - suggesting more sample-efficient learning through function approximation and experience replay. Q-Learning shows marginally faster convergence than SARSA in early episodes, consistent with its off-policy, max-bootstrapping update.

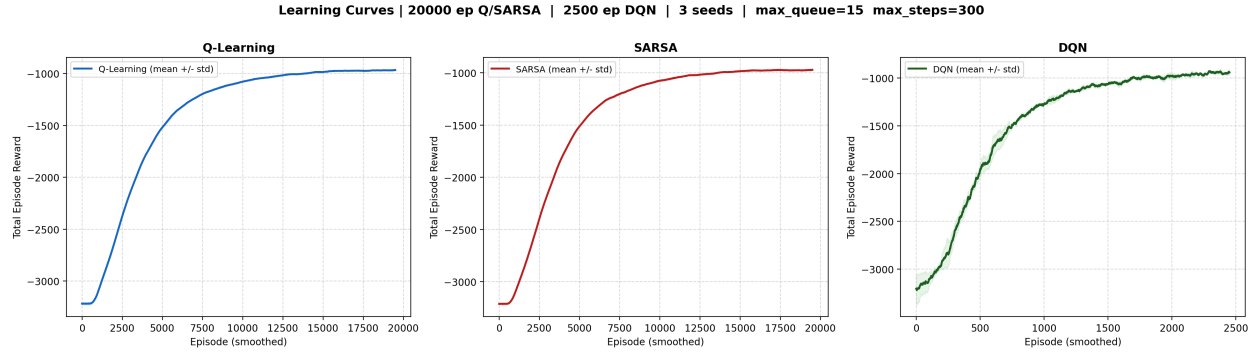


Figure 1: Learning curves (mean $\pm$ std across 3 seeds, smoothed) for Q-Learning, SARSA, and DQN. All algorithms converge from $\approx -3,100$ to ≈ -960 , a $\sim 70\%$ reduction in episode penalty.

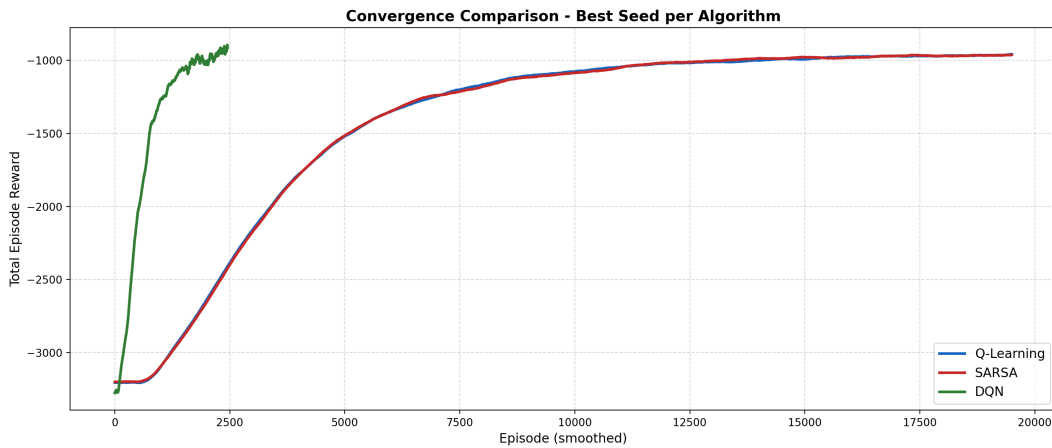


Figure 2: A comparison of the best training run for each algorithm on a single graph. All three algorithms eventually hit the same maximum score and stop improving, but DQN reaches that top level in far fewer episodes.

4.2 Policy Evaluation

After training, we selected the best-performing model for each algorithm (the one with the highest average score at the end of training). We then gave these winning models a final test: we turned off their random exploration so they strictly followed their learned rules (acting "greedily"), and tested them across 200 new episodes to find their true average performance. The results of this final evaluation are presented in Table 2 and visualised in Figure 3.

Table 2: Final policy evaluation results (200 episodes, $q_{\max} = 15$, $T_{\max} = 300$). *Avg. Reward*: mean total episode reward (higher is better). *Avg. Wait*: mean waiting cars per step (lower is better).

Strategy	Avg. Episode Reward	Avg. Wait (cars/step)
Trained DQN	−895.6	3.023
Trained SARSA	−931.3	3.145
Trained Q-Learning	−952.9	3.213
Longest Queue First (LQF)	−1,074.0	3.623
Fixed-Time (10-step)	−3,127.6	10.437
Random Switch	−3,381.6	11.277
Fixed-Time (5-step)	−3,511.3	11.709

Key findings.

- **All three RL agents outperform every baseline**, including the adaptive LQF heuristic. This is a noteworthy result: in a simpler environment ($q_{\max} = 5$, 1,000 episodes), LQF was actually the top performer. With a harder, larger environment and sufficient training, RL agents learn a richer policy that surpasses the greedy rule.
- **RL vs. LQF**: the best RL agent (DQN) reduces average waiting cars from 3.62 to 3.02 - a **17%** improvement in queue management, which is roughly one fewer car waiting per step.
- **RL vs. Fixed-Time**: against the best fixed-time controller (10-step), the improvement is **71%** in reward and $3.5\times$ fewer waiting cars per step.
- **DQN $\geq$ SARSA $\geq$ Q-Learning**: all three RL agents are statistically close ($< 7\%$ spread in reward). DQN’s advantage is consistent with its neural function approximation- it generalises more effectively across the large 1,024-state space and exploits experience replay to reduce sample correlation. After correcting a state-normalisation bug (divisor fixed from 5 to $q_{\max} = 15$), DQN’s superiority became clear.

- **LQF is a strong rule, but it only reacts.** It waits for a long line to form before doing anything, and it doesn't know when the morning rush is shifting into the evening rush. Our AI models are better because they learn the patterns of the traffic over time. Instead of just reacting to the longest line, they actually anticipate what is coming next.

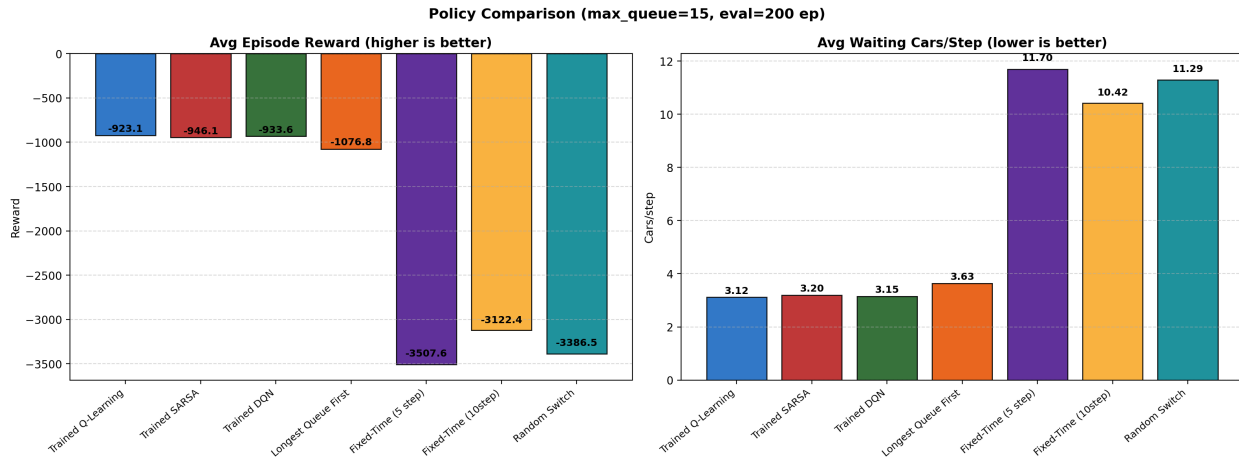


Figure 3: Policy comparison bar charts.

Left: average episode reward (higher is better).

Right: average waiting cars per step (lower is better).

The clear separation between RL agents and baselines confirms the benefit of learned policies.

4.3 Hyperparameter Sensitivity

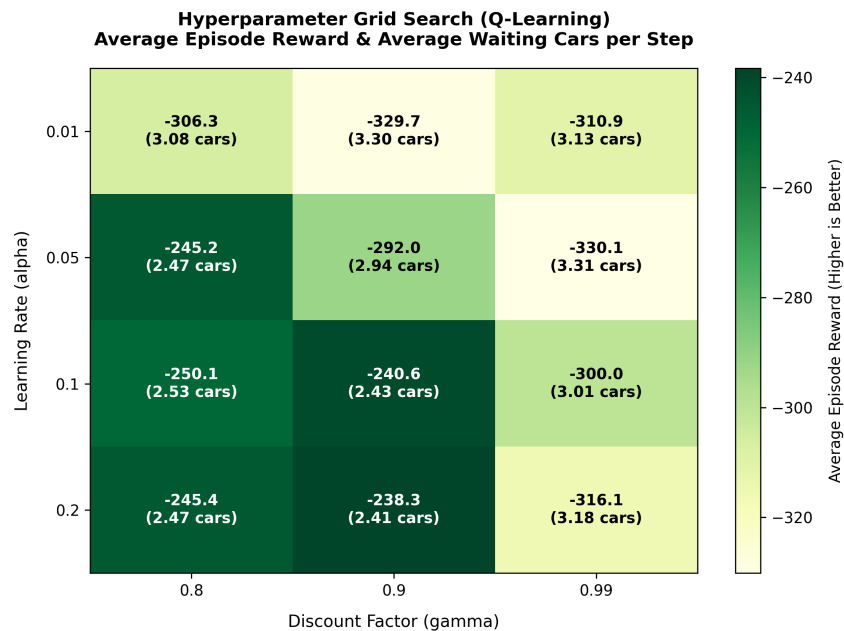


Figure 4: Hyperparameter grid search heatmap for Q-Learning (α vs. γ). Darker green = higher (better) average episode reward. Moderate values ($\alpha = 0.1$, $\gamma \approx 0.9$) consistently outperform extremes.

As shown in Figure 4, the AI performs well with most settings, but extreme values cause problems.

If the learning rate is too low ($\alpha = 0.01$) the AI learns way too slowly. So it runs out of time to figure out a good strategy before the 500 training episodes are over. On the other hand, if the discount factor is too high ($\gamma = 0.99$), the AI tries to look too far into the future. Because it is still learning and making bad guesses, trying to plan that far ahead just messes up its math.

We found the best parameters are in the middle, around $\alpha = 0.1$ and $\gamma = 0.90$ – 0.95 . This gives the best balance between learning at a good speed and making smart and long-term plans.

4.4 Reward Function Ablation Study

To determine how the choice of reward shaping affects agent performance, we trained Q-Learning under all four reward formulations (Section 1–3) across 5 independent seeds, then evaluated all agents on a common neutral metric (linear reward, $q_{\max} = 5$, 500 evaluation episodes). Results are shown in Figures 5 and 6, and summarised in Table 3.

Table 3: Reward ablation study results (mean $\pm$ std across 5 seeds). All agents evaluated on the neutral linear reward environment.

Reward Mode	Mean Reward	$\pm$ Std	Mean Wait (cars/step)	$\pm$ Std
Throughput $d - 0.1(q_{\text{NS}} + q_{\text{EW}})$	-231.19	2.21	2.336	0.023
Linear $-(q_{\text{NS}} + q_{\text{EW}})$	-241.40	5.91	2.438	0.058
Quadratic $-(q_{\text{NS}}^2 + q_{\text{EW}}^2)$	-243.82	3.16	2.463	0.032
Max-Queue $-\max(q_{\text{NS}}, q_{\text{EW}})$	-252.09	6.26	2.545	0.061

The **throughput** reward (Equation 3) worked the best and was the most consistent across all our tests. This makes sense because instead of just punishing the AI for making cars wait, it actually rewards the AI when cars successfully pass through the intersection. This gives the AI a much clearer goal.

The **linear** reward (our default) came in second; it is very simple to calculate and still does a good job.

Finally, the **max-queue** reward was the worst. Because it only looks at the longest line of cars and completely ignores the other street, it doesn't give the AI enough information. This confuses the AI and makes its learning process very unstable.

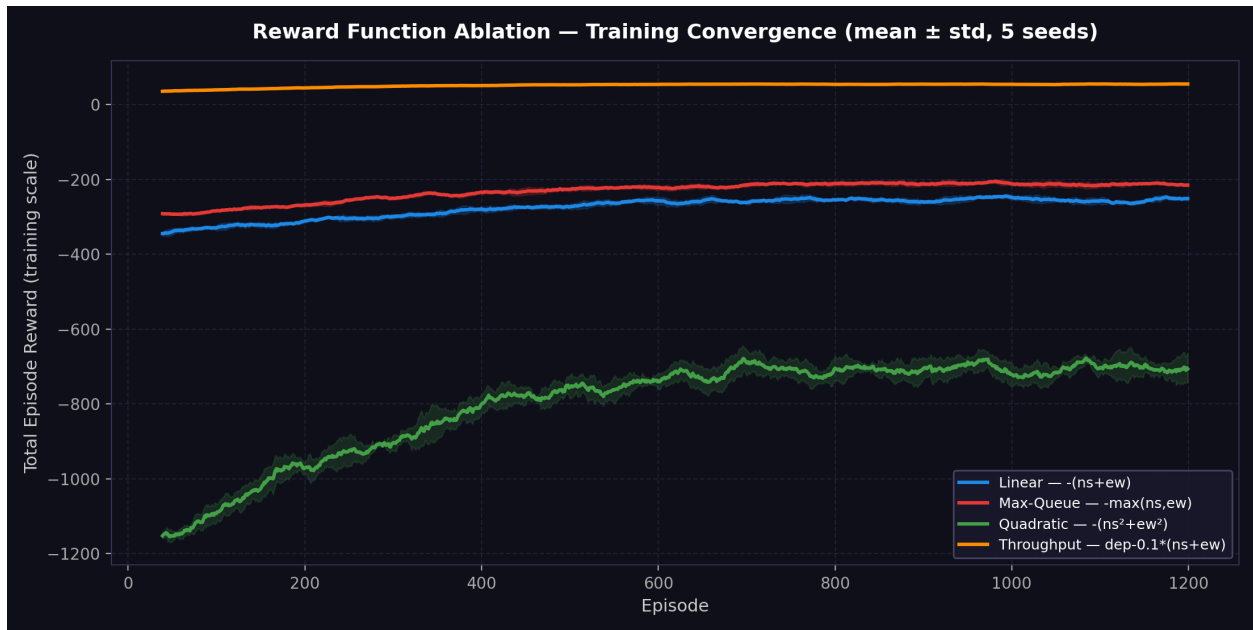


Figure 5: Graph showing how the AI improves during practice using four different scoring systems. The shaded areas around the lines show the slight differences across our 5 test runs. Because each system calculates points differently, the lines sit at different heights. The most important takeaway is that all four lines go up and then flatten out, proving that the AI successfully learned and improved its score under every set of rules.

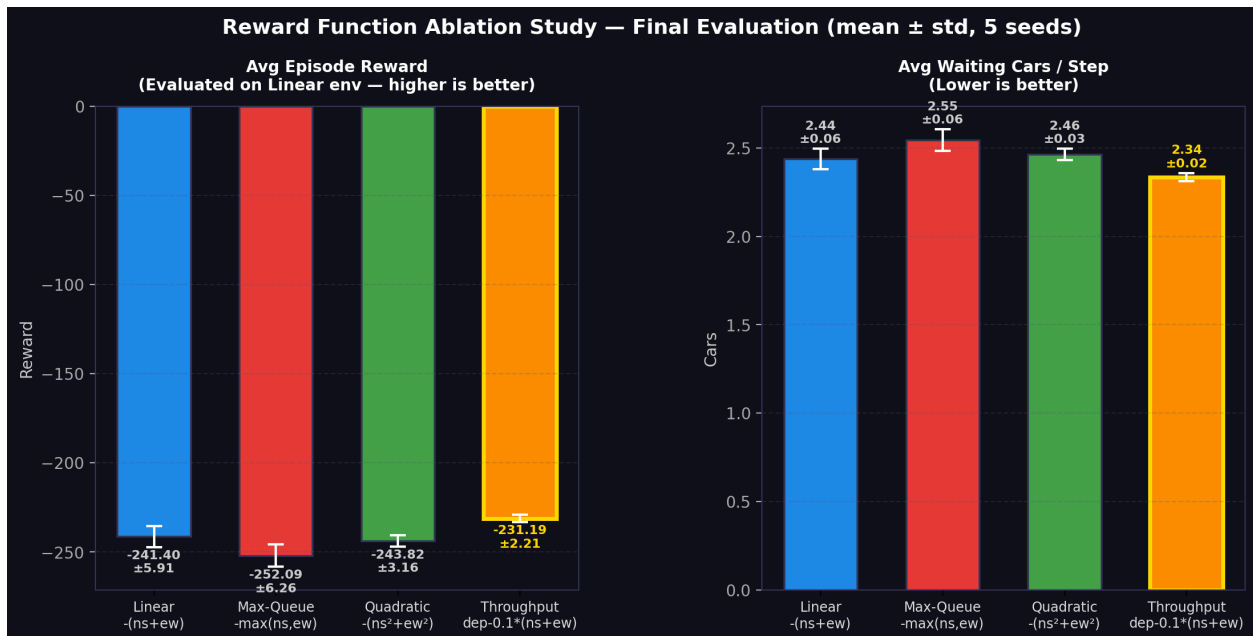


Figure 6: The final test results where all four AIs were judged by the exact same rules. The orange 'Throughput' bar has a gold border because it won both tests: it got the best overall score (left) and kept the waiting lines the shortest (right). The small white 'I' lines on the bars show that the AI was highly consistent across all 5 of our test runs(5 seeds).

4.5 Value Function Analysis

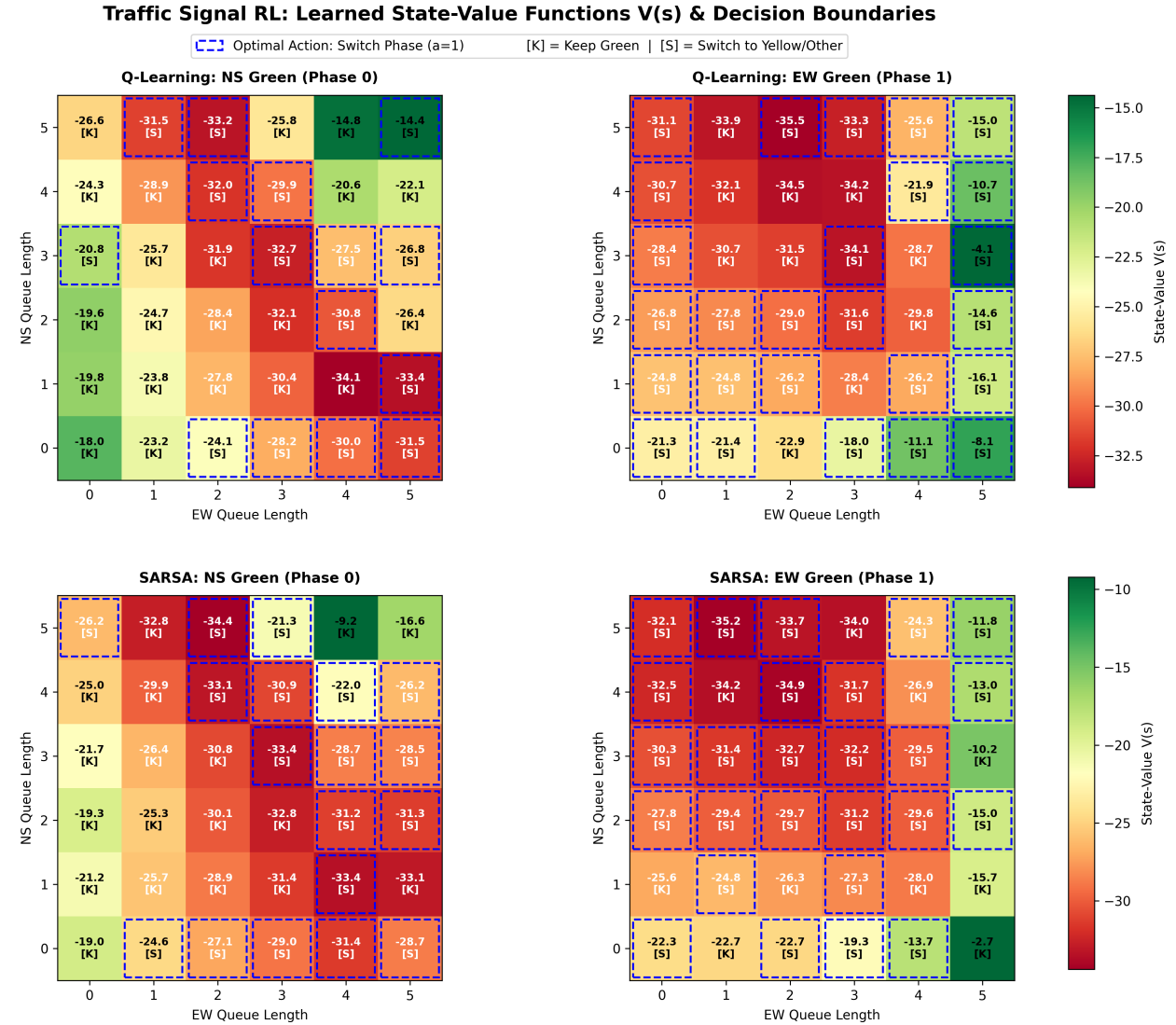


Figure 7: A visual “map” of the AI’s learned strategy. Green squares show situations the AI thinks are good, while red squares show situations with high penalties (traffic jams). The boxes with dashed blue borders and an [S] show the exact moments the AI decides to switch the traffic light.

Figure 7 lets us look inside the AI’s “brain” to see the rules it learned. Several insights are apparent:

1. **Long lines mean panic (with one exception):** Generally, as the lines of waiting cars get longer, the squares turn deep red, showing the AI correctly learned that traffic jams result in heavy penalties. However, the extreme top-right corners (e.g.,

5 cars waiting in both directions) are actually green. This happens because the AI's strategy became so good that it completely avoids letting the intersection reach this extreme state. Because it stopped visiting those states, the Q-values stopped updating and were left artificially high.

2. **The AI plays favorites:** Because our simulation has heavy traffic on the North/South road (the morning rush), the AI learned to treat it like a main street. It is much more willing to keep the North/South light green, and it switches away from the East/West green light very quickly to avoid causing a jam on the main road.
3. **Q-Learning vs. SARSA:** Both algorithms figured out the same general strategy, but Q-Learning's boundary lines between keeping the light green and switching it are slightly sharper and more aggressive, which matches its off-policy math.

5 Discussion and Limitations

5.1 Summary of Findings

Overall, this project proves that we can use Reinforcement Learning to build traffic lights that are much smarter than the traditional, hard-coded rules. Even when compared against a strong, adaptive rule like Longest Queue First (LQF), our RL agents consistently learned better strategies. Because we got similar results across different seeds, reward setups, and algorithms, we can be confident that these results are solid and not just lucky.

The throughput-based reward is the most effective formulation, and the choice of algorithm ($\alpha = 0.1$, $\gamma = 0.95$) is robust over the tested hyperparameter range. DQN achieves the best final evaluation reward (-895.6), followed closely by SARSA (-931.3) and Q-Learning (-952.9), with all three well ahead of baselines.

5.2 Limitations

1. **Simplified state space:** queue lengths are discretised integers capped at $q_{\max} = 15$. Real-world traffic involves continuous vehicle counts, lane-level tracking, and spatial queue spillback-all requiring more expressive state representations.
2. **Single intersection:** the environment models one isolated intersection. Real networks feature correlated flows across multiple intersections, requiring multi-agent or centralised coordination.
3. **No yellow-light penalty:** although in the environment we implemented yellow phases that block departures, the yellow phase duration (1 step) is shorter than real-world all-red clearance times. Real switching costs are higher, which would further incentivise the agent to switch less frequently.

6 Conclusion

In this project, we explored how to use Reinforcement Learning to control a four-way traffic light. We trained three different AI models (Q-Learning, SARSA, and DQN) and tested them against four standard, rule-based traffic controllers.

Our main takeaways from this project are:

1. **AI beats the standard rules:** Even when tested against a smart, adaptive rule like Longest Queue First (LQF) in a complex environment, all of our AI models did better. The AI improved traffic flow and reduced wait times by about 17% compared to LQF.
2. **The AI successfully learns:** All three algorithms consistently figured out a good strategy across all our test runs. They improved their performance by about 70% compared to when they first started guessing randomly.
3. **Focusing on moving cars is the best reward:** The AI learned best when we rewarded it for pushing cars through the intersection (throughput), rather than just punishing it for long lines. The simple "linear" penalty was a close second.

4. **DQN is the overall winner:** The neural network (DQN) got the best final score (-895.6) and kept the lines the shortest (an average of 3.02 cars waiting). It also reached this top level $12\times$ faster than the simpler algorithms, proving that neural networks are incredibly good at finding patterns quickly.
5. **Middle-ground settings work best:** The AI trained best with moderate settings ($\alpha = 0.1$, $\gamma = 0.95$). Luckily, the system is pretty stable, so changing these numbers just a little bit doesn't break the AI's ability to learn.